Hubpieces — diagramme de classes et guide de lecture

Version du document : 13 octobre 2026 Références vérifiées : les 13 entités annotées `@Entity` sous `src/main/java/com/hubpieces/model/`, les relations JPA, et le dump `src/main/resources/static/DB/hubpieces_db.sql` (MariaDB). Le parcours métier est recoupé avec les services et contrôleurs Java.

> **Document prêt à copier dans Google Docs.** Le PDF compagnon est `docs/EXPLICATION-DIAGRAMME-CLASSES.pdf`; il contient l'explication et le code Mermaid, sans rendu graphique automatique. Le code Mermaid ci-dessous est conservé en source; si le diagramme n'est pas rendu dans Google Docs, suivez les étapes de la section « Insérer Mermaid dans Google Docs ».

1. Vue d’ensemble

Le schéma contient **13 entités JPA** et leurs **13 tables métier**. Il contient aussi la table SQL `utilisateur_role`, générée par le `@ManyToMany` de `Utilisateur` : cette table d’association n’est pas une quatorzième entité Java. Au total, le dump décrit **14 tables**.

Les entités décrivent les données persistées : comptes, demandes, fichiers réels, offres, paiements, échanges, avis et litiges. Les statuts sont des énumérations stockées dans des colonnes, et non des tables séparées.

2. Diagramme Mermaid complet — code source

Source Mermaid — bloc complet à copier dans Mermaid Live Editor

```
classDiagram
    direction LR

    class Utilisateur {
        +Long id
        +String nom
        +String prenom
        +String email
        +String mot_de_passe
        +String telephone
        +String ville
        +String code_postal
        +Double latitude
        +Double longitude
        +Double note_moyenne
        +Integer nombre_avis
        +StatutUtilisateur statut
        +LocalDateTime date_inscription
        +String photo_profil
    }

    class Role {
        +Long id
        +RoleNom nom
    }

    class UtilisateurRole {
        <<table associative>>
        +Long utilisateur_id
        +Long role_id
    }

    class ProfilMaker {
        +Long id
        +Long utilisateur_id
        +Integer rayon_action_km
        +String materiaux_disponibles
        +String description_imprimante
        +Integer nombre_commandes_actives
        +Integer nombre_avis_positifs
        +BadgeMaker badge
    }

    class ProfilExpertCao {
        +Long id
        +Long utilisateur_id
        +String logiciels_maitrise
        +String portfolio_url
```

```
    +String description
}

class Demande {
    +Long id
    +String titre
    +String description
    +String marque
    +String modele_appareil
    +String materiau_souhaite
    +Double budget_max
    +String ville
    +String code_postal
    +Double latitude
    +Double longitude
    +EtatDemande etat
    +LocalDateTime date_creation
    +LocalDateTime date_modification
    +Long demandeur_id
    +Long fichier_stl_id
}

class FichierStl {
    +Long id
    +String nom
    +String description
    +String chemin_fichier
    +Long taille_fichier
    +String categorie
    +String tags
    +String marque
    +String modele_appareil
    +String materiau_recommande
    +String dimensions
    +Double note_moyenne
    +Integer nombre_telechargements
    +EtatValidation etat_validation
    +LocalDateTime date_upload
    +Long uploadeur_id
    +Long valideur_id
}

class PhotoDemande {
    +Long id
    +Long demande_id
    +String chemin_photo
    +LocalDateTime date_upload
}
```

```
class OffreMaker {
    +Long id
    +Long demande_id
    +Long maker_id
    +Double prix
    +Integer delai_jours
    +String message
    +ModeLivraison mode_livraison
    +EtatOffre etat
    +LocalDateTime date_creation
}

class Message {
    +Long id
    +Long demande_id
    +Long expediteur_id
    +Long destinataire_id
    +String contenu
    +Boolean lu
    +LocalDateTime date_envoi
}

class Transaction {
    +Long id
    +Long offre_maker_id
    +Double montant_total
    +Double commission_plateforme
    +Double montant_maker
    +Double montant_expert_cao
    +EtatPaiement etat_paiement
    +LocalDateTime date_paiement
    +LocalDateTime date_liberation
    +LocalDateTime date_creation
}

class Avis {
    +Long id
    +Long transaction_id
    +Long auteur_id
    +Long destinataire_id
    +Integer note
    +String commentaire
    +Boolean publie
    +LocalDateTime date_creation
    +LocalDateTime date_publication
}
```

```
    class Litige {
        +Long id
        +Long transaction_id
        +Long demandeur_id
        +String raison
        +EtatLitige etat
        +String decision
        +LocalDateTime date_ouverture
        +LocalDateTime date_resolution
        +Long admin_id
    }

    class Notification {
        +Long id
        +Long utilisateur_id
        +String titre
        +String message
        +TypeNotification type
        +Boolean lu
        +LocalDateTime date_creation
    }

    class RoleNom {
        <<enumeration>>
        DEMANDEUR
        MAKER
        EXPERT_CAO
        ADMIN
    }
    class StatutUtilisateur {
        <<enumeration>>
        ACTIF
        SUSPENDU
        EN_ATTENTE
    }
    class EtatDemande {
        <<enumeration>>
        NOUVELLE
        MODELISATION_REQUISE
        RECHERCHE_MAKER
        ATTENTE_IMPRESSION
        EN_COURS_IMPRESSION
        ATTENTE_VALIDATION
        ANNULEE
        CLOTUREE
    }
    class EtatValidation {
        <<enumeration>>
```

```
        EN_ATTENTE
        VALIDE
        REJETE
    }
    class EtatOffre {
        <<enumeration>>
        EN_ATTENTE
        ACCEPTEE
        REFUSEE
        ANNULEE
    }
    class EtatPaiement {
        <<enumeration>>
        EN_ATTENTE
        PAYE
        EN_SEQUESTRE
        LIBERE
        REMBOURSE
        LITIGE
    }
    class EtatLitige {
        <<enumeration>>
        OUVERT
        EN_COURS
        RESOLU
        FERME
    }
    class ModeLivraison {
        <<enumeration>>
        REMISE_EN_MAIN
        LIVRAISON_POSTALE
    }
    class BadgeMaker {
        <<enumeration>>
        DEBUTANT
        CONFIRME
        EXPERT
    }
    class TypeNotification {
        <<enumeration>>
        INFO
        SUCCESS
        WARNING
        ERROR
    }

    Utilisateur "1" <-- "0..*" UtilisateurRole : utilisateur_id PK/FK
    Role "1" <-- "0..*" UtilisateurRole : role_id PK/FK
```

```
    Utilisateur "1" <-- "0..1" ProfilMaker : utilisateur_id FK UNIQUE NOT NULL
    Utilisateur "1" <-- "0..1" ProfilExpertCao : utilisateur_id FK UNIQUE NOT NULL

    Utilisateur "1" <-- "0..*" Demande : demandeur_id FK NOT NULL
    Demande "0..*" o-- "0..1" FichierStl : fichier_stl_id FK NULL
    Demande "1" *-- "0..*" PhotoDemande : demande_id FK / cascade ALL + orphanRemoval
    Demande "1" *-- "0..*" OffreMaker : demande_id FK / cascade ALL + orphanRemoval
    Demande "1" *-- "0..*" Message : demande_id FK / cascade ALL + orphanRemoval

    Utilisateur "1" <-- "0..*" OffreMaker : maker_id FK NOT NULL
    Utilisateur "1" <-- "0..*" FichierStl : uploadeur_id FK NOT NULL
    Utilisateur "0..1" <-- "0..*" FichierStl : valideur_id FK NULL
    Utilisateur "1" <-- "0..*" Message : expediteur_id FK NOT NULL
    Utilisateur "1" <-- "0..*" Message : destinataire_id FK NOT NULL
    Utilisateur "1" <-- "0..*" Notification : utilisateur_id FK NOT NULL
    Utilisateur "1" <-- "0..*" Avis : auteur_id FK NOT NULL
    Utilisateur "1" <-- "0..*" Avis : destinataire_id FK NOT NULL
    Utilisateur "1" <-- "0..*" Litige : demandeur_id FK NOT NULL
    Utilisateur "0..1" <-- "0..*" Litige : admin_id FK NULL

    OffreMaker "1" <-- "0..1" Transaction : offre_maker_id FK UNIQUE NOT NULL
    Transaction "1" <-- "0..*" Avis : transaction_id FK NOT NULL
    Transaction "1" <-- "0..*" Litige : transaction_id FK NOT NULL
```

Le code ci-dessus est la source fidèle. Ce PDF ne contient pas le diagramme Mermaid rendu en image.

Le diagramme montre volontairement `UtilisateurRole` pour rendre visible la table d'association SQL. Il ne faut pas en déduire qu'une classe Java `UtilisateurRole` existe.

3. Légende : PK, FK, cardinalités et unicité

- **PK (Primary Key / clé primaire)** : identifie une ligne de façon unique. Les 13 tables d'entités ont une PK `id` générée par identité (`AUTO_INCREMENT` dans le dump). `utilisateur_role` a une PK composée de `(utilisateur_id, role_id)`.
- **FK (Foreign Key / clé étrangère)** : référence la PK d'une autre table. Elle garantit qu'un lien pointe vers une ligne existante, mais ne prouve pas à elle seule que l'utilisateur possède un rôle particulier.
- `NOT NULL` : la relation correspondante est obligatoire pour la ligne enfant. Une FK nullable (`NULL`) rend le lien facultatif.

- **Multiplicité près d'une classe** : nombre d'instances de cette classe liées à une instance de l'autre côté. `1` = exactement une, `0..1` = aucune ou une, `0..*` = zéro ou plusieurs.
- `UNIQUE` : interdit deux lignes ayant la même valeur pour la colonne ou le groupe de colonnes concerné; c'est ce qui transforme certaines relations de « plusieurs » en « au plus une ».

Unicités visibles dans le SQL réel

1. `utilisateur.email` est `UNIQUE` : deux comptes ne peuvent pas partager la même adresse.
2. `profil_maker.utilisateur_id` et `profil_expert_cao.utilisateur_id` sont chacun `UNIQUE NOT NULL` : chaque profil appartient à un compte, et un compte possède au plus un profil de chaque type.
3. `transaction.offre_maker_id` est `UNIQUE NOT NULL` : une transaction porte sur une offre, et une offre est liée à au plus une transaction.
4. La PK composite de `utilisateur_role` empêche d'attribuer deux fois le même rôle au même compte.
5. Dans le dump, `role.nom` n'a pas de contrainte `UNIQUE`; le fait que les quatre rôles seedés aient des identifiants distincts n'est pas une unicité SQL sur ce champ.
6. `demande.fichier_stl_id` n'est ni `UNIQUE` ni `NOT NULL` : une demande peut n'avoir aucun fichier, et le même fichier validé peut être référencé par plusieurs demandes.
7. Les FKs `fichier_stl.valideur_id` et `litige.admin_id` sont facultatives. Les autres FKs du diagramme sont requises par le dump.

4. Les 13 entités JPA, une par une

Entité Java (table SQL)	Ce qu'elle représente	Liens importants
Utilisateur (`utilisateur`)	Compte, coordonnées, statut, note et photo de profil. `email` est unique.	Porte plusieurs rôles via `utilisateur_role`; peut demander, modéliser, fabriquer, envoyer/recevoir des messages, déposer/recevoir des avis, recevoir des notifications et ouvrir/gérer des litiges selon ses droits applicatifs.
Role (`role`)	Un des rôles stockés (`DEMANDEUR`, `MAKER`, `EXPERT_CAO`, `ADMIN`).	Plusieurs comptes peuvent avoir le même rôle; les rôles d'un compte sont reliés par `utilisateur_role`.
Demande (`demande`)	Besoin de réparation : description de la pièce, marque/modèle, matériau, budget, lieu, état et dates.	Obligatoirement un demandeur; éventuellement un `FichierStl`; contient photos, offres et messages en composition JPA.
FichierStl (`fichier_stl`)	Métadonnées d'un vrai fichier de catalogue ou d'une modélisation/demande : chemin, taille, description, catégorie, matériau et validation.	Un uploader obligatoire et un valideur facultatif, tous deux des `Utilisateur`; une demande peut le référencer si son état le permet.
PhotoDemande (`photo_demande`)	Chemin et date d'une photo envoyée pour documenter une demande.	Appartient à une seule demande; enfant géré en composition par `Demande`.
OffreMaker (`offre_maker`)	Prix, délai, message, livraison et état de la proposition faite pour une demande.	Une demande et un Maker obligatoires; éventuellement une transaction après acceptation.
ProfilMaker (`profil_maker`)	Rayon d'action, matériaux, imprimante, statistiques et badge du Maker.	Un compte unique, au plus un profil Maker par compte.
ProfilExpertCao (`profil_expert_cao`)	Logiciels maîtrisés, portfolio et description de l'Expert CAO.	Un compte unique, au plus un profil Expert CAO par compte.
Message (`message`)	Contenu, date et état lu/non lu d'un échange.	Une demande, un expéditeur et un destinataire obligatoires; enfant géré en composition par `Demande`.
Notification (`notification`)	Titre, texte, type, date et état lu/non lu d'une notification.	Un utilisateur destinataire obligatoire.
Transaction (`transaction`)	Montants total, Maker, commission, part CAO, dates et état du paiement.	Une seule offre Maker obligatoire; plusieurs avis ou litiges peuvent s'y rattacher.
Avis (`avis`)	Note (contrainte SQL de 1 à 5), commentaire, auteur, destinataire et publication.	Un auteur, un destinataire et une transaction obligatoires. Le SQL n'impose pas une seule ligne d'avis par auteur/transaction.
Litige (`litige`)	Raison, état, décision et dates de résolution d'un désaccord.	Une transaction et un demandeur obligatoires; l'administrateur qui le traite est facultatif jusqu'à sa prise en charge.

Comptage : 13 classes Java `@Entity`, listées ci-dessus. Les enums (rôles, états, badge, livraison, type de notification) sont des valeurs de colonnes et ne sont pas des entités supplémentaires.

5. `utilisateur_role` : la table d'association des rôles

Dans `Utilisateur.java`, `roles` est déclaré avec `@ManyToMany` et `@JoinTable(name = "utilisateur_role", ...)`. Le dump SQL confirme les deux colonnes `NOT NULL`, leurs FKs et la clé primaire `(utilisateur_id, role_id)`.

- Un utilisateur peut donc avoir plusieurs rôles.
- Un même rôle peut être attribué à plusieurs utilisateurs.
- Une ligne représente **un couple utilisateur–rôle**.
- Il n'existe pas de classe JPA `UtilisateurRole` dans le code inspecté : Hibernate/JPA gère la table d'association.
- La table ne contient pas de colonne `id` autonome et ne porte pas de données supplémentaires (date d'attribution, auteur, etc.).

Les quatre valeurs d'énumération du rôle sont `DEMANDEUR`, `MAKER`, `EXPERT_CAO` et `ADMIN`. Le rattachement d'un fichier à `uploadeur_id`, ou d'une offre à `maker_id`, référence un compte générique; c'est la couche applicative (session/contrôleur/service) qui vérifie les droits, pas une FK SQL vers un « utilisateur Maker » distinct.

6. Comment lire les associations et les cardinalités

Lien (FK portée par la table enfant)	Lecture métier	Justification réelle
`demande.demandeur_id` → `utilisateur.id`	Un demandeur peut avoir 0 à N demandes; chaque demande a exactement 1 demandeur.	FK `NOT NULL`, `@ManyToOne`.
`demande.fichier_stl_id` → `fichier_stl.id`	Une demande a 0 ou 1 fichier; un fichier peut être référencé par 0 à N demandes.	FK nullable, non unique, `@ManyToOne` sans cascade.
`photo_demande.demande_id` → `demande.id`	Une demande a 0 à N photos; chaque photo a exactement 1 demande.	FK `NOT NULL`, `@OneToMany` côté demande avec cascade et suppression des orphelins.
`offre_maker.demande_id` → `demande.id`	Une demande a 0 à N offres; chaque offre concerne exactement 1 demande.	FK `NOT NULL`; collection `Demande.offres` en composition.
`offre_maker.maker_id` → `utilisateur.id`	Un compte Maker peut déposer 0 à N offres; chaque offre a exactement 1 Maker.	FK `NOT NULL`. Le rôle est vérifié par la logique applicative.
`fichier_stl.uploadeur_id` / `valideur_id` → `utilisateur.id`	Un compte peut téléverser ou valider 0 à N fichiers; chaque fichier a 1 uploader et 0 ou 1 valideur.	Uploader `NOT NULL`; valideur nullable; les deux sont des `@ManyToOne`.
`message.demande_id` → `demande.id`	Une demande a 0 à N messages; chaque message appartient à 1 demande.	FK `NOT NULL`; composition JPA depuis `Demande`.
`message.expediteur_id` / `destinataire_id` → `utilisateur.id`	Chaque message a 1 expéditeur et 1 destinataire; un compte peut participer à plusieurs messages dans chaque sens.	Deux FKs `NOT NULL` vers `utilisateur`.
`notification.utilisateur_id` → `utilisateur.id`	Un compte reçoit 0 à N notifications; chaque notification a 1 destinataire.	FK `NOT NULL`.
`avis.transaction_id`, `auteur_id`, `destinataire_id`	Chaque avis porte sur 1 transaction et a 1 auteur et 1 destinataire; chacun peut être lié à plusieurs avis.	Trois FKs `NOT NULL`, aucune unicité transaction/auteur dans le dump.
`litige.transaction_id`, `demandeur_id`, `admin_id`	Chaque litige concerne 1 transaction et 1 demandeur; il peut être traité par 0 ou 1 admin.	Transaction et demandeur `NOT NULL`, admin nullable.
`profil_maker.utilisateur_id` / `profil_expert_cao.utilisateur_id`	Un compte a 0 ou 1 profil de chacun de ces types; chaque profil a 1 compte.	FK `NOT NULL` et `UNIQUE` dans chaque table de profil.
`transaction.offre_maker_id` → `offre_maker.id`	Une offre a 0 ou 1 transaction; chaque transaction est attachée à exactement 1 offre.	FK `NOT NULL UNIQUE`; relation JPA `@OneToOne`.
`utilisateur_role`	Un utilisateur et un rôle peuvent chacun figurer dans 0 à N lignes d'association.	Deux FKs `NOT NULL`, PK composite. C'est la matérialisation du N–N.

`0..1` signifie « facultatif, au plus un »; ce n'est pas la promesse qu'une ligne existe. Par exemple, `UNIQUE` sur `transaction.offre_maker_id` garantit au plus une transaction par offre, mais n'oblige pas chaque offre à avoir une transaction.

7. Composition, agrégation et associations simples

Composition — losange plein *--

`Demande` compose ses `PhotoDemande`, `OffreMaker` et `Message`. Dans `Demande.java`, ces trois collections `@OneToMany` utilisent `cascade = CascadeType.ALL` et `orphanRemoval = true`. Au niveau de l'application JPA/Hibernate, les changements de cycle de vie de la demande sont propagés à ces enfants; retirer un enfant de la collection peut entraîner sa suppression.

Nuance importante : le dump SQL ne déclare pas `ON DELETE CASCADE`. Le losange plein représente la règle JPA, pas une clause de cascade ajoutée aux FKs dans le schéma SQL. Une suppression faite directement en SQL n'exécute pas automatiquement `CascadeType.ALL`.

Agrégation — losange ouvert o--

`Demande` peut référencer un `FichierStl`, mais ne le possède pas. Le lien est un `@ManyToOne` sans cascade; sa FK est nullable et non unique. Le fichier peut exister avant la demande et être partagé. Supprimer une demande ne doit pas faire disparaître un modèle du catalogue.

Associations simples — flèches --> ou <--

Les liens vers un utilisateur (Maker, uploader, valideur, expéditeur, destinataire, auteur, destinataire d'avis, demandeur de litige, admin), les profils, `Transaction`–`OffreMaker`, avis, litiges, notifications et rôles sont des associations de référence. Une FK, même obligatoire, n'en fait pas à elle seule une composition. La seule relation à plusieurs-à-plusieurs est celle des rôles, matérialisée par `utilisateur_role`.

8. Rôles et principaux états métier

Rôles

- **DEMANDEUR** : décrit la pièce, suit sa demande, choisit une offre, confirme la réception et peut ouvrir un litige.
- **EXPERT_CAO** : modélise une pièce lorsque la demande l'exige et transmet le vrai fichier pour validation.
- **ADMIN** : valide ou rejette les fichiers soumis et arbitre les litiges.
- **MAKER** : propose une offre pour une demande publiée, puis fabrique la pièce si son offre est acceptée.

Un compte peut porter plusieurs rôles via la table de jointure; le diagramme ne suppose pas un compte distinct par métier.

États persistés (colonnes ENUM)

Enum Java / colonne	Valeurs observées	Sens métier
`StatutUtilisateur` / `utilisateur.statut`	`ACTIF`, `SUSPENDU`, `EN_ATTENTE`	État du compte.
`EtatDemande` / `demande.etat`	`NOUVELLE`, `MODELISATION_REQUISE`, `RECHERCHE_MAKER`, `ATTENTE_IMPRESSION`, `EN_COURS_IMPRESSION`, `ATTENTE_VALIDATION`, `ANNULEE`, `CLOTUREE`	Avancement global de la demande.
`EtatValidation` / `fichier_stl.etat_validation`	`EN_ATTENTE`, `VALIDE`, `REJETE`	Décision de validation d'un fichier réel.
`EtatOffre` / `offre_maker.etat`	`EN_ATTENTE`, `ACCEPTEE`, `REFUSEE`, `ANNULEE`	Cycle d'une proposition Maker.
`EtatPaiement` / `transaction.etat_paiement`	`EN_ATTENTE`, `PAYE`, `EN_SEQUESTRE`, `LIBERE`, `REMBOURSE`, `LITIGE`	Suivi de la somme de la transaction.
`EtatLitige` / `litige.etat`	`OUVERT`, `EN_COURS`, `RESOLU`, `FERME`	Traitement du désaccord.

Les autres enums sont également des valeurs de colonnes, pas des tables : `RoleNom` (`DEMANDEUR`, `MAKER`, `EXPERT_CAO`, `ADMIN`), `ModeLivraison` (`REMISE_EN_MAIN`, `LIVRAISON_POSTALE`), `BadgeMaker` (`DEBUTANT`, `CONFIRME`, `EXPERT`) et `TypeNotification` (`INFO`, `SUCCESS`, `WARNING`, `ERROR`). JPA utilise `@Enumerated(EnumType.STRING)` pour les états concernés; le dump SQL les matérialise comme colonnes `ENUM`.

`NOUVELLE` est la valeur par défaut de l'entité/colonne `Demande`, mais le service de création affecte explicitement `MODELISATION_REQUISE` sans fichier validé, ou `RECHERCHE_MAKER` si le demandeur choisit un `FichierStl` déjà `VALIDE`.

9. Cycle réel : DEMANDEUR → EXPERT_CAO → ADMIN → MAKER

1. **DEMANDEUR — crée le besoin.** Une `Demande` est créée avec `demandeur_id`. Sans fichier déjà validé, le service la place en `MODELISATION_REQUISE`. Si le demandeur sélectionne un vrai fichier du catalogue déjà `VALIDE`, la demande peut aller directement en `RECHERCHE_MAKER`.
2. **EXPERT_CAO — modélise et téléverse.** Pour une demande en `MODELISATION_REQUISE`, l'expert dépose un `FichierStl` avec l'état `EN_ATTENTE` et l'associe à la demande. La demande reste en attente de modélisation pendant la décision administrative; la file CAO n'expose pas un doublon pendant qu'un fichier attend.
3. **ADMIN — accepte ou rejette le vrai fichier.** À l'acceptation, `fichier_stl.etat_validation` devient `VALIDE`, le valideur est renseigné et la demande associée passe à `RECHERCHE_MAKER`. Au rejet, le fichier passe à `REJETE` et est détaché de la demande; celle-ci reste à modéliser.
4. **MAKER — propose la fabrication.** Les offres ne peuvent être créées que pour une demande en `RECHERCHE_MAKER`. Plusieurs makers peuvent proposer une `OffreMaker` (le service empêche un même maker d'en déposer plusieurs pour la même demande). Le demandeur choisit une offre : elle devient `ACCEPTEE`, les autres offres encore en attente sont refusées, la demande passe à `ATTENTE_IMPRESSION` et une `Transaction` est créée en `EN_SEQUESTRE`.
5. **Fabrication et réception.** Le maker accepté fait passer la demande à `EN_COURS_IMPRESSION`, puis à `ATTENTE_VALIDATION` lorsqu'il annonce la pièce prête. Le demandeur clôture après réception (`CLOTUREE`); le paiement est alors libéré (`LIBERE`).
6. **Litige éventuel.** Le demandeur peut créer un `Litige` relatif à la transaction; le paiement passe à `LITIGE` pendant l'instruction. L'admin enregistre sa décision. Dans le service actuel, un litige clôturé rembourse si le texte de décision contient « rembours »; sinon le paiement est libéré. Les états du litige et du paiement restent deux colonnes distinctes.

Fichiers réellement téléversés : `FichierStlService.uploadAndCreate` accepte les extensions STL, OBJ, 3MF, STEP et STP (50 Mo maximum dans ce chemin). Les visuels GLB ci-dessous ne sont pas ces fichiers de fabrication.

10. Les visuels GLB décoratifs ne sont pas des entités

Les fichiers `repair-gear.glb`, `printed-piece.glb`, `cao-model.glb`, `circuit-board.glb`, `admin-shield.glb`, `eco-recycle.glb` et `auth_blob.glb` sont des ressources statiques de présentation, servies depuis `static/images/` et affichées avec `<model-viewer>`.

- Ils ne sont **ni des classes Java ni des tables SQL**.
- Ils ne génèrent **aucune ligne** `FichierStl` et ne sont pas présentés comme téléchargeables ou fabricables.
- `circuit-board.glb` est uniquement le visuel du hero de `catalogue.html`; il n'est pas ajouté à `FALLBACK_STL` ni au catalogue métier par défaut.
- `cao-model.glb` illustre l'espace Expert CAO; il ne prétend pas être le résultat d'une demande.
- `repair-gear.glb` et `printed-piece.glb` sont des décorations de dashboard, pas des demandes ou commandes réelles.
- `auth_blob.glb` est le même asset réutilisé sur `login.html` et `register.html`, sans copie supplémentaire.

Les arrière-plans décoratifs sont CSS; les `<model-viewer>` concernés ont un fond transparent, un texte alternatif et un chargement différé. La rotation automatique respecte `prefers-reduced-motion`.

11. Insérer Mermaid dans Google Docs

Google Docs ne transforme pas nativement un bloc Mermaid en diagramme. Le code source ci-dessus est conservé entre les délimiteurs ``` `mermaid `` et `` ` ```.

1. Copier uniquement le contenu du bloc Mermaid, sans les lignes de clôture.
2. Ouvrir **Mermaid Live Editor** et coller le code.
3. Vérifier le rendu puis exporter le diagramme en **PNG** (ou SVG si le flux d'import Google Docs le permet).
4. Dans Google Docs, choisir **Insertion → Image → Importer depuis l'ordinateur** et sélectionner le PNG.
5. Garder aussi la source Mermaid dans le document ou en annexe pour pouvoir modifier le diagramme ultérieurement.